

DIBUJITOS



Dibujitos

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de Prueba	3
6. Ejemplos	3

1. Equipo

Nombre	Apellido	Legajo	E-mail
Juan Ignacio	Cantarella	64509	jcantarella@itba.edu.ar
Máximo Daniel	Carranza	64538	mcarranza@itba.edu.ar
Alex	Köhler	64320	akohler@itba.edu.ar
Javier	Liu	64332	jaliu@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [Dibujitos](#).

3. Dominio

El proyecto consiste en el desarrollo de un domain specific language (DSL) que permite crear dibujos 2D a partir de figuras geométricas y expresiones matemáticas.

El objetivo del lenguaje es obtener una sintaxis simple, pero expresiva, con la que los usuarios podrán definir diseños, patrones y figuras. El lenguaje ofrecerá la posibilidad de asignarle nombres a puntos, para que estos puedan ser reutilizados.

La salida del programa será en formato SVG. Esto asegura compatibilidad con navegadores web, editores gráficos y otras herramientas que soporten el formato, y también impone una limitación sobre el tipo de dibujos que pueden generarse, acotando el DSL a aquellas construcciones que puedan expresarse en el mismo.

Este DSL se ubica en la intersección entre la programación y el diseño visual, y puede ser útil en contextos educativos, artísticos o técnicos donde se requiera expresar gráficamente una idea.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones y funcionalidades:

- (I). Las variables podrán ser de tipo int o floats (el default sería float).
- (II). Las variables también podrán ser vectores compuestos por 2 floats.
- (III). Se podrán crear arreglos estáticos con datos del mismo tipo.
- (IV). Se proveerán operaciones aritméticas básicas como +, -, * y / para números.
- (V). Se proveerán operaciones aritméticas sobre vectores 2D, como suma, resta, proyección, distancia y multiplicación o división por un entero o float.
- (VI). Se proveerán las estructuras de control IF-ELSE, FOR y FOR-EACH, pero no estará permitido tener ciclos infinitos.
- (VII). Se proveerán operadores relacionales como <, >, =, ≠, ≤ y ≥.
- (VIII). Se proveerán operaciones lógicas básicas como AND, OR y NOT.
- (IX). Se podrán definir líneas poligonales y polígonos mediante una secuencia de puntos.
- (X). Se podrán definir círculos con un punto y un radio.
- (XI). Se podrá definir curvas mediante curvas de Bézier.
- (XII). Se podrá “modificar el estilo”, es decir:
 - (XII.1). Color y opacidad de línea.
 - (XII.2). Color y opacidad de relleno.
 - (XII.3). Grosor de línea.
 - (XII.4). Orden Z (si la figura está adelante o atrás).
 - (XII.5). Unión de línea (linejoin).
- (XIII). Habrán algunas constantes predefinidas, como YELLOW (color amarillo), INVISIBLE (opacidad 0), FRONT (z layer más alta).
- (XIV). Se podrá agregar comentarios de línea con //.
- (XV). Imprimir texto por consola

5. Casos de Prueba

Se proponen los siguientes casos iniciales de prueba de **aceptación**:

- (I). Un programa que cree un punto mediante *operaciones con enteros* y dibuje un círculo con el centro en ese punto.
- (II). Un programa que cree un punto mediante *operaciones con vectores* y dibuje un círculo con el centro en ese punto.
- (III). Un programa que *dibuje un círculo con borde negro y relleno amarillo*.
- (IV). Un programa que tenga comentarios.
- (V). Un programa que use un FOR-EACH para imprimir en consola números del 1 al 10.
- (VI). Un programa que dibuje un polígono con un FOR-EACH.
- (VII). Un programa que dibuje una única curva de Bézier con forma de S.
- (VIII). Un programa que dibuje un corazón usando curvas de Bézier.
- (IX). Un programa que dibuje muchos círculos cada uno con un estilo diferente.
- (X). Un programa que dibuje un fractal finito de forma algorítmica con un FOR.
- (XI). Modificar el orden Z (FRONT / BACK) de diferentes figuras y verificar su renderizado.
- (XII). Dibujar una figura sin relleno (fill(INVISIBLE)) y con línea visible.
- (XIII). Dibujar una figura compuesta (casa: cuadrado + triángulo + puerta + ventana).

Además, los siguientes casos de prueba de **rechazo**:

- (I). Un programa malformado: paréntesis desbalanceados

```
circle(-5,3), 10);
```

- (II). Un programa malformado: estructura for sin iterador

```
for i in {  
    circle((0,0), 10);  
}
```

- (III). Un programa que use una variable o función que no existe.
- (IV). Un programa que produzca división por cero.
- (V). Un programa que intente operar entre tipos de datos incompatibles.
- (VI). Indexación fuera de rango en un arreglo (acceder a arr[10] cuando sólo hay 5 elementos).
- (VII). Uso de palabras reservadas como nombres de variables (circle = 5).
- (VIII). Un programa que pasa la cantidad incorrecta de parámetros en la función circle.

6. Ejemplos

Operaciones con puntos (o vectores) y distancias:

```
// definir puntos
p1 = (25, 0);
p2 = (50, 0);
// definir distancias
d1 = 50;

// imprimir en consola
log("Value: ", d1, "\n");

// definir arreglos
v1 = [1, 2, 3, 4, 5, 1000];

// operaciones
p5 = p1 + p2;      // suma de puntos/vectores
p6 = p2 - p1;      // resta de puntos/vectores
d3 = 5 + d1;       // suma de distancias (es como usar ints)
d5 = p1.x;         // proyección
p8 = (p1.x, p2.x); // creo nuevo vector a partir de 2 anteriores
d4 = dist(p8);     // sqrt( p8.x ^2 + p8.y ^2 )
p7 = p1/2;         // p1.x/2 ; p1.y/2

// Crear círculos
circle(p7, d1);    // Centro p7, radio d1
```

Dibujar una carita feliz

```
// definir estilo
fill(YELLOW);
stroke(3);
  circle((0;0), 10);

fill(000000h);
stroke(3);
  circle((5,3), 10);
  circle((-5,3), 10);
  curve((5,-3),(0,-6),(-5,-3));
```

Dibujar círculos concéntricos de cada vez menor tamaño, alternando color.

```
for i in [1:10] {  
  if (i % 2 == 1) {  
    fill(YELLOW);  
  }  
  else {  
    fill(BLACK);  
  }  
  circle((0,0), 100/i);  
}
```

Dibujar el fractal “Sierpiński triangle” hasta profundidad 2.

```
// Determinar tamaño del triángulo  
size = 100;  
half = size / 2;  
height = size * sqrt(3)/2;  
  
// Estilo: solamente borde negro sin relleno  
stroke(BLACK);  
fill(INVISIBLE);  
  
// Recorrer las tres filas (profundidad=2)  
for row in [0:2] {  
  // En cada fila, dibujar (row + 1) triángulos  
  for col in [0:row] {  
    // Computar el punto de la izquierda-abajo del triángulo  
    x = half - (row * half) + col * size;  
    y = row * height;  
  
    // Definir los tres puntos del triángulo  
    p1 = (x, y);  
    p2 = (x + half, y + height);  
    p3 = (x + size, y);  
  
    polygon(p1, p2, p3);  
  }  
}
```